09-01 성능 측정 및 벤치마킹 시스템 분석

개요

Solana는 고성능 블록체인으로서 정확한 성능 측정과 벤치마킹이 매우 중요합니다. 이 문서에서는 Solana의 성능 측정 도구와 TPS(Transactions Per Second) 벤치마킹 시스템의 구현을 상세히 분석합니다.

1. Measure 모듈 - 기본 성능 측정 도구

1.1 Measure 구조체

`measure/src/measure.rs` 의 핵심 구조체는 다음과 같습니다:

```rust
#[derive(Debug)]
pub struct Measure {
name: &'static str,
start: Instant,
duration: u64,
}
```

주요 필드:

- `name` : 측정 작업의 이름 (정적 문자열)
- `start` : 측정 시작 시점 (`Instant` 타입)
- `duration` : 측정된 지속 시간 (나노초 단위)

1.2 핵심 메서드

측정 시작

```
pub fn start(name: &'static str) -> Self {
Self {
name,
start: Instant::now(),
duration: 0,
}
}
```

측정 종료

```
pub fn stop(&mut self) {
self.duration = duration_as_ns(&self.start.elapsed());
}
```

다양한 시간 단위 변환

```
pub fn as_ns(&self) -> u64 { self.duration }
pub fn as_us(&self) -> u64 { self.duration / 1000 }
pub fn as_ms(&self) -> u64 { self.duration / (1000 * 1000) }
pub fn as_s(&self) -> f32 {
self.duration as f32 / (1000.0f32 * 1000.0f32 * 1000.0f32)
```

```
}
```

즉시 종료 및 반환 메서드

```
pub fn end_as_ns(self) -> u64 { duration_as_ns(&self.start.elapsed()) }
pub fn end_as_us(self) -> u64 { duration_as_us(&self.start.elapsed()) }
pub fn end_as_ms(self) -> u64 { duration_as_ms(&self.start.elapsed()) }
pub fn end_as_s(self) -> f32 { duration_as_s(&self.start.elapsed()) }
```

1.3 Display 구현

```
impl fmt::Display for Measure {
fn fmt(&self, f: &mut fmt::Formatter) -> fmt::Result {
if self.duration == 0 {
write!(f, "{} running", self.name)
} else if self.as_us() < 1 {
write!(f, "{} took {}ns", self.name, self.duration)
} else if self.as_ms() < 1 {
write!(f, "{} took {}us", self.name, self.as_us())
} else if self.as_s() < 1. {
write!(f, "{} took {}ms", self.name, self.as_ms())
} else {
write!(f, "{} took {:.1}s", self.name, self.as_s())
```

```
}
}
}
```

특징:

- 자동으로 적절한 시간 단위 선택
- 나노초부터 초까지 가독성 있는 출력
- 실행 중인 상태도 표시

2. Bench-TPS 시스템 - TPS 벤치마킹 도구

2.1 시스템 아키텍처

Bench-TPS는 Solana 네트워크의 처리량을 측정하는 종합적인 벤치마킹 도구입니다.

주요 컴포넌트:

1. **Transaction Generator**: 트랜잭션 생성
2. **Sender Threads**: 트랜잭션 전송
3. **Sampler Thread**: TPS 샘플링
4. **Performance Monitor**: 성능 모니터링

2.2 Configuration 구조체

```rust
#[derive(PartialEq, Debug)]
pub struct Config {
pub json_rpc_url: String,
```

```
pub websocket_url: String,
pub id: Keypair,
pub threads: usize,
pub duration: Duration,
pub tx_count: usize,
pub keypair_multiplier: usize,
pub thread_batch_sleep_ms: usize,
pub sustained: bool,
pub external_client_type: ExternalClientType,
pub use_quic: bool,
pub tpu_connection_pool_size: usize,
pub compute_unit_price: Option<ComputeUnitPrice>,
// ... 기타 설정들
}
```

2.3 벤치마킹 프로세스

메인 벤치마킹 함수

```
pub fn do_bench_tps<T>(
client: Arc<T>,
config: Config,
gen_keypairs: Vec<Keypair>,
nonce_keypairs: Option<Vec<Keypair>>,
```

```
) -> u64
where
T: 'static + BenchTpsClient + Send + Sync + ?Sized,
```

주요 단계:

1. 초기 트랜잭션 카운트 획득
2. 샘플링 스레드 시작
3. 송신 스레드들 생성
4. 트랜잭션 생성 및 전송
5. 성능 통계 수집 및 보고

2.4 TPS 샘플링 시스템

SampleStats 구조체

```
#[derive(Default)]
pub struct SampleStats {
/// Maximum TPS reported by this node
pub tps: f32,
/// Total time taken for those txs
pub elapsed: Duration,
/// Total transactions reported by this node
pub txs: u64,
}
```

샘플링 함수

```
pub fn sample_txs<T>(
exit_signal: Arc<AtomicBool>,
sample_stats: &Arc<RwLock<Vec<(String, SampleStats)>>>,
sample_period: u64,
client: &Arc<T>,
) where
T: BenchTpsClient + ?Sized,
```

샘플링 로직:

1. 주기적으로 트랜잭션 카운트 조회
2. 이전 샘플과의 차이 계산
3. TPS 계산: `sample_txs / elapsed_time`
4. 최대 TPS 추적

2.5 트랜잭션 생성 시스템

TransactionChunkGenerator

```
struct TransactionChunkGenerator<'a, 'b, T: ?Sized> {
client: Arc<T>,
account_chunks: KeypairChunks<'a>,
nonce_chunks: Option<KeypairChunks<'b>>,
chunk_index: usize,
```

```
reclaim_lamports_back_to_source_account: bool,
compute_unit_price: Option<ComputeUnitPrice>,
instruction_padding_config: Option<InstructionPaddingConfig>,
skip_tx_account_data_size: bool,
}
```

트랜잭션 생성 메서드

```
fn generate(&mut self, blockhash: Option<&Hash>) -> Vec<TimestampedTransaction
let tx_count = self.account_chunks.source.len();
info!(
"Signing transactions... {} (reclaim={}, blockhash={:?})",
tx_count, self.reclaim_lamports_back_to_source_account, blockhash
);
let signing_start = Instant::now();
// 트랜잭션 생성 로직...
let duration = signing_start.elapsed();
let ns = duration.as_secs() * 1_000_000_000 + u64::from(duration.subsec_nanos()
let bsps = (tx_count) as f64 / ns as f64;
let nsps = ns as f64 / (tx_count) as f64;
info!(
"Done. {:.2} thousand signatures per second, {:.2} us per signature, {} ms total
bsps * 1_000_000_f64,
nsps / 1_000_f64,
```

```
duration_as_ms(&duration),
);
}
```

2.6 성능 최적화 기법

병렬 트랜잭션 서명

```
pairs_with_compute_unit_prices
.par_iter()
.map(|((from, to), compute_unit_price)| {
(
transfer_with_compute_unit_price_and_padding(
from,
&to.pubkey(),
1,
*blockhash,
instruction_padding_config,
Some(**compute_unit_price),
skip_tx_account_data_size,
),
Some(timestamp()),
)
})
```

```
.collect()
```

배치 전송 최적화

```
const FUND_CHUNK_LEN: usize = 4 * 1024 * 1024 / 512; // 4MB 네트워크 버퍼 고려
```

2.7 클라이언트 타입별 성능 특성

ExternalClientType 열거형

```
#[derive(Eq, PartialEq, Debug)]
pub  enum ExternalClientType {
// RPC 클라이언트를 통한 트랜잭션 제출
RpcClient,
// TPU 클라이언트를 통한 직접 리더 전송
TpuClient,
}
```

성능 차이점:

- **RpcClient**: 안정적이지만 상대적으로 낮은 처리량
- **TpuClient**: 높은 처리량, 직접 리더와 통신

2.8 Compute Unit Price 최적화

ComputeUnitPrice 설정

```
#[derive(Debug, PartialEq)]
pub enum ComputeUnitPrice {
Fixed(u64), // 고정 가격
Random, // 랜덤 가격 (0..50 * 1000 범위)
}
```

우선순위 수수료 계산

```
pub fn max_lamports_for_prioritization(compute_unit_price: &Option<ComputeUnit
let Some(compute_unit_price) = compute_unit_price else {
return 0;
};

let compute_unit_price = match compute_unit_price {
ComputeUnitPrice::Random => (MAX_RANDOM_COMPUTE_UNIT_PRICE as u128)
.saturating_mul(COMPUTE_UNIT_PRICE_MULTIPLIER as u128),
ComputeUnitPrice::Fixed(compute_unit_price) => *compute_unit_price as u128,
};

const MICRO_LAMPORTS_PER_LAMPORT: u64 = 1_000_000;
let micro_lamport_fee: u128 =
compute_unit_price.saturating_mul(TRANSFER_TRANSACTION_COMPUTE_UNIT as u128);
let fee = micro_lamport_fee
```

```
.saturating_add(MICRO_LAMPORTS_PER_LAMPORT.saturating_sub(1) as u128)

.saturating_div(MICRO_LAMPORTS_PER_LAMPORT as u128);

u64::try_from(fee).unwrap_or(u64::MAX)

}
```

3. 성능 측정 메트릭

3.1 주요 성능 지표

1. **TPS (Transactions Per Second)**

- 초당 처리된 트랜잭션 수
- 네트워크 처리량의 핵심 지표

2. **서명 속도 (Signatures Per Second)**

- 초당 생성된 디지털 서명 수
- 클라이언트 측 성능 지표

3. **지연 시간 (Latency)**

- 트랜잭션 제출부터 확인까지의 시간
- 사용자 경험의 핵심 요소

4. **처리량 (Throughput)**

- 단위 시간당 처리된 데이터량
- 네트워크 용량 지표

3.2 성능 모니터링 구현

실시간 TPS 모니터링

```
info!(
"Sampler {:9.2} TPS, Transactions: {:6}, Total transactions: {} over {} s",
tps,
sample_txs,
total_txs,
total_elapsed.as_secs(),
);
```

메트릭 데이터 포인트

```
datapoint_info!(
"bench-tps-generate_txs",
("duration", duration_as_us(&duration), i64)
);
```

4. 벤치마킹 모드

4.1 Peak Mode vs Sustained Mode

Peak Mode (기본값)

- 최대 성능 측정에 집중
- 트랜잭션 생성과 전송을 순차적으로 실행
- 더 높은 순간 TPS 달성 가능

Sustained Mode

```
if  sustained {
// 지속적인 성능 모드
while  shared_txs.read().unwrap().len() > 2 * threads {
sleep(Duration::from_millis(1));
}
} else {
// 피크 성능 모드
while !shared_txs.read().unwrap().is_empty()
|| shared_tx_active_thread_count.load(Ordering::Relaxed) > 0
{
sleep(Duration::from_millis(1));
}
}
```

4.2 QUIC vs UDP 성능 비교

QUIC 연결 (기본값)

```
ConnectionCache::new_quic(
"bench-tps-connection_cache_quic",
tpu_connection_pool_size,
)
```

UDP 연결

```
ConnectionCache::with_udp(
"bench-tps-connection_cache_udp",
tpu_connection_pool_size,
)
```

성능 특성:

- **QUIC**: 더 안정적인 연결, 약간의 오버헤드
- **UDP**: 낮은 지연시간, 패킷 손실 가능성

5. 성능 최적화 전략

5.1 병렬 처리 최적화

1. **Rayon을 활용한 병렬 서명**
2. **멀티스레드 트랜잭션 전송**
3. **비동기 I/O 활용**

5.2 메모리 최적화

1. **트랜잭션 풀 관리**
2. **키페어 재사용**
3. **배치 크기 조정**

5.3 네트워크 최적화

1. **연결 풀 크기 조정**

2. **배치 전송 최적화**

3. **적응적 백오프 전략**

결론

Solana의 성능 측정 및 벤치마킹 시스템은 다음과 같은 특징을 가집니다:

1. **정밀한 시간 측정**: 나노초 단위의 정확한 성능 측정

2. **종합적인 벤치마킹**: TPS, 지연시간, 처리량 등 다양한 지표 측정

3. **확장 가능한 아키텍처**: 멀티스레드 및 병렬 처리 지원

4. **실시간 모니터링**: 지속적인 성능 추적 및 보고

5. **유연한 설정**: 다양한 테스트 시나리오 지원

이러한 도구들을 통해 Solana는 지속적으로 성능을 모니터링하고 최적화할 수 있으며, 개발자들은 네트워크의 실제 성능을 정확히 측정할 수 있습니다.